

Interview Question

C1. Behavioural Interview Questions (STAR Format)

1. Tell me about yourself.

Situation: Software Engineering Honours student at Charles Darwin University, currently working full-time as a Software Developer at Radical Systems building ASP.NET Core/Vue.js applications, and in this course project moved from a developer role into Tester/QA.

Action: Over the last several weeks I've built a full testing practice from the ground up — manual test planning, Selenium/API automation, CI/CD integration, performance and security testing — plus I already had hands-on QA experience from a prior unit testing an open-source document management system (Mayan EDMS).

Result: That combination of real development experience and dedicated QA practice means I understand both sides of the pipeline — what's easy to break, and how to catch it before it ships.

2. Describe a challenging situation you faced.

Situation: While converting a Selenium test suite into BDD scenarios with Gherkin/behave, one scenario needed to test an empty username field.

Action: The scenario kept erroring out instead of failing normally — I dug into behave's step-matching and found its default parser can't match an empty string inside quoted parameters, so I switched that step file to the regex matcher instead.

Result: All three scenarios passed correctly afterward, and it's now something I specifically watch for when writing parameterised BDD steps.

3. Tell me about a time you worked under pressure.

Situation: At Radical Systems, a production bug was reported affecting an existing feature.

Action: I reproduced the issue, traced it through the ASP.NET Core backend, wrote a fix, and added NUnit regression tests so the same bug couldn't silently reappear.

Result: The fix shipped the same day and the added coverage caught a related edge case before it reached users again.

4. How do you manage multiple priorities?

Situation: I'm working full-time as a developer while completing this course's testing practice, which spans manual testing, automation, CI/CD, and security testing across multiple weeks.

Action: I break work into small, independently verifiable pieces — one tool or technique at a time — and I don't mark anything "done" until I've actually run it and have evidence (screenshots, CI logs) that it works.

Result: That discipline meant every piece of practice work stayed genuinely reliable, rather than a backlog of half-finished claims.

5. Tell me about a conflict within a team.

Situation: In a prior unit's group project testing Mayan EDMS, module boundaries between "Ingestion and Classification" (my module) and other members' modules weren't always clear.

Action: I raised it early, we agreed on clear ownership boundaries and shared test data conventions, and I documented my module's scope explicitly in the report.

Result: This avoided duplicated or conflicting test coverage and made it clear which findings belonged to which module during report review.

6. Describe a mistake you made.

Situation: Early on, a written practice plan suggested putting a new test tool's setup directly inside another week's shared test folder.

Action: I realised that would break that week's already-working CI pipeline the moment it touched a file that pipeline depended on, so I separated the new work into its own self-contained folder instead.

Result: Both pipelines kept working independently, and it's now how I approach any change to a shared file — check what else depends on it first.

7. Tell me about a time you adapted to change.

Situation: Partway through this course project I moved from a developer role into Tester/QA.

Action: I had to quickly build testing-specific skills — manual test design, Selenium/API automation, CI integration, performance and security testing — on top of my existing development background.

Result: My prior development experience actually made the transition faster, since I already understood what the systems under test were doing internally.

8. Describe a time when you demonstrated leadership.

Situation: While preparing my resume during this course, I noticed a description of a testing tool that wasn't quite accurate to what I'd actually used.

Action: Rather than leave a convenient but slightly misleading phrase, I corrected it to accurately reflect the real tool and reasoning behind the choice.

Result: It's a small example, but it reflects how I approach QA generally — I'd rather flag an inaccuracy myself than have someone else catch it later.

9. Tell me about a time you influenced someone.

Situation: During the Mayan EDMS project, my automated test suite caught that the API allowed creating duplicate cabinets, even though the UI correctly blocked it.

Action: I confirmed the bug independently with a raw curl request (removing any doubt it was a test artifact) and reported it clearly with reproduction steps.

Result: The finding was accepted as a genuine defect in the group's report — a good example of automation catching something manual testing alone might have missed.

10. Why should we hire you?

Situation: The role needs someone who can both write good tests and understand the system well enough to know what's worth testing.

Action: I bring real development experience, hands-on QA experience across manual testing, automation, CI/CD, performance, and security testing, and a track record of only calling something "done" once I've actually verified it.

Result: I can contribute to test coverage and quality confidence from day one, without needing to learn the fundamentals on the job.

C2. Tester / QA Technical Questions

1. Walk me through how you approach writing a test plan and test cases.

Situation: For the SauceDemo login feature, I started with a test plan defining scope, entry/exit criteria, and environment.

Action: I wrote 15 manual test cases using equivalence partitioning and boundary value analysis — covering functional (valid login), negative (wrong password, empty fields), and boundary (very long input, whitespace) cases.

Result: That structured approach caught real defects I could log and track, and gave me a clear base to decide which cases were worth automating later.

2. What's the difference between Severity and Priority?

Answer:

- Severity = technical impact (how badly it breaks the system), usually set by the tester.
- Priority = business urgency (how soon it needs fixing), usually set by the product owner.
- They move independently — a typo in a footer is low severity, but high priority right before a big client demo.
- I've worked with both fields in Jira, and specifically noted when a project only had Priority configured, since it's easy to conflate the two if only one field exists.

3. How do you decide what to automate versus keep manual?

Answer:

- I follow the automation pyramid — automate stable, repetitive, frequently-run tests low down (unit/API), and keep exploratory or rarely-run edge cases manual.
- In practice: I automated 5 of 15 manual login test cases with Selenium (the ones worth re-running on every change), wrote a separate Postman suite for API-level checks (faster and more stable than UI tests), and left one-off exploratory testing manual.

4. Walk me through a bug you found and how you reported it.

Situation: While testing an open-source document management system's ingestion module, I built a Robot Framework suite covering the cabinet-creation API.

Action: The suite caught that creating a duplicate cabinet name wasn't rejected by the API, even though the UI blocked it — I confirmed it independently with a raw curl request to rule out a test-script issue.

Result: I documented it as a real defect with reproduction steps and expected vs actual behaviour — exactly the kind of gap between UI and API validation that's easy to miss if you only test through the UI.

5. What's the difference between SAST and DAST?

Answer:

- SAST (static) scans source code without running it — finds issues like unsafe functions or hardcoded secrets early, shift-left.

- DAST (dynamic) attacks the running application from the outside, like a real attacker, no code access needed.
- I've used Bandit (SAST) and OWASP ZAP (DAST) together on the same codebase — Bandit flagged a risky code pattern, ZAP found a deprecated dependency and confirmed things from the outside that static analysis alone wouldn't catch.

6. How do you approach performance/load testing?

Situation: I've run JMeter load tests at both a smaller scale (50 concurrent users against a public REST endpoint) and a larger scale (100 and 400 concurrent users against a real application's endpoints).

Action: I look at throughput, average/min/max response time, and error rate together — not just one number — and dig into View Results Tree when error rates aren't 0%.

Result: At 400 users I found a CPU-bound bottleneck and a ~4% error rate on one endpoint that didn't show up at all at lower concurrency — the kind of issue functional testing alone would never catch.

7. What is BDD and why use it alongside traditional test automation?

Answer:

- BDD writes tests in plain Given/When/Then language (Gherkin) that both technical and non-technical stakeholders can read and agree on.
- It sits on top of existing automation, not instead of it — my Gherkin scenarios call into the same Page Object Model classes as my pytest suite; only the description layer changes.
- The output itself reads like living documentation, which is genuinely useful for showing non-technical people what's actually being tested.

8. How do you make sure your CI pipeline actually catches regressions, not just that it looks like it should?

Situation: After wiring Selenium tests running against a multi-browser Grid into GitHub Actions, I didn't want to just assume the pipeline would fail on a real regression.

Action: I deliberately broke a test's assertion, pushed it, and confirmed the build actually went red — then reverted and confirmed it went back to green.

Result: That before/after pair is real proof the gate works, rather than just trusting that a non-zero exit code "should" fail the build.

9. Tell me about a security testing exercise you've done.

Situation: Practicing against OWASP Juice Shop (a deliberately vulnerable app built for this purpose), I combined manual exploration with an automated OWASP ZAP scan.

Action: Manually, I confirmed a SQL-injection login bypass and a reflected XSS in the search bar (a real alert actually fired); the ZAP scan separately flagged a Content-Security-Policy misconfiguration.

Result: Between the two approaches I had three real, reproducible findings — manual testing caught what automated scanning can't (chained logic), and the scanner caught a passive misconfiguration I wasn't specifically looking for.

10. Why should we hire you as a tester?

Answer:

- I bring both manual and automated testing skills across UI, API, performance, and security — not just one narrow specialty.
- I have real development background, so I understand what's happening under the hood, not just what the UI shows.
- I don't call anything "done" without evidence — screenshots, CI logs, actual re-runs — which means what I report is something a team can actually trust.